

Introduction

The goal of this project was to build a robot that takes input from a keyboard and writes it with a marker. The logic was achieved through a custom-built Central Processing Unit (CPU) in Verilog with assembly (MIPS) code that was uploaded into an FPGA. The movement in the x and y axes was made possible through a two-axis motion enabler that leveraged rollers into two rails. It was equipped with two stepper motors for the two axes, and the whole thing was provided by Chris Bingham. A servo with a rack and pinion was mounted on the x-axis stepper motor to allow control in the z-axis. An expo whiteboard marker was used to write characters on a white board.

Specifications

- Length: 56 cm
- Width: 40 cm
- Height: 30 cm
- Character Size: 4.5 cm x 4.5 cm
- Line Length: 6 characters
- Board Length: 6 lines
- Clock Frequency: 25 MHz
- Stepper Motors: NEMA 17
- Motor Drivers: DRV8825

CPU

The CPU was custom-designed in Verilog as a 5-staged pipelined processor (Fetch, Decode, Execute, Memory, Write). Data and control hazards were mitigated through bypassing. Through extensive clock testing on Vivado, the maximum operating frequency of the CPU was determined to be 25MHz. Thus, a clocking wizard was used to generate a 25MHz clock for the CPU, given the 100MHz clock that the FPGA had internally.

No custom modifications were made to the CPU. However, if there were more time, a modification that would help would be adding a move instruction that would move the motor based on register values provided. This would simplify the MIPS code significantly. Another could be implementing a beq instruction that would make the MIPS programming easier to understand, though it does work with the bne instructions.

Inputs/Outputs

The primary input into the FPGA was the keyboard. The keyboard functions by sending unique scan codes. When the key is pressed and held, it sends out its unique scan code. These scan codes can be used to determine which key was pressed.

There are three main outputs for the FPGA, which are the two stepper motors that control the x and y-directions and the servo that controls the z-direction.

The stepper motors are controlled by the DRV8825 stepper motor drivers. The schematic of the circuit used to control the stepper motor drivers is displayed in Figure 1. A1 and A2, along with B1 and B2, are the outputs of the driver to control the motor coils. The inputs to the motor driver are simplified, only requiring a step and a direction, which were JA[1] and JA[2] for the x-direction and JA[7] and JA[8] for the y-direction. Microstepping was available via the mode pins, but they were unused. The driver itself was powered with 3.3V, and 12V was input into the driver to power the motor. A 47 μ F capacitor was placed between the 12V and GND power pins to ease large voltage spikes. The built-in potentiometer was tuned to 0.8V to limit the current. The RESET' and SLEEP' pins were powered, while the ENABLE' pin was left unpowered to allow inputs to the driver to always be changed.

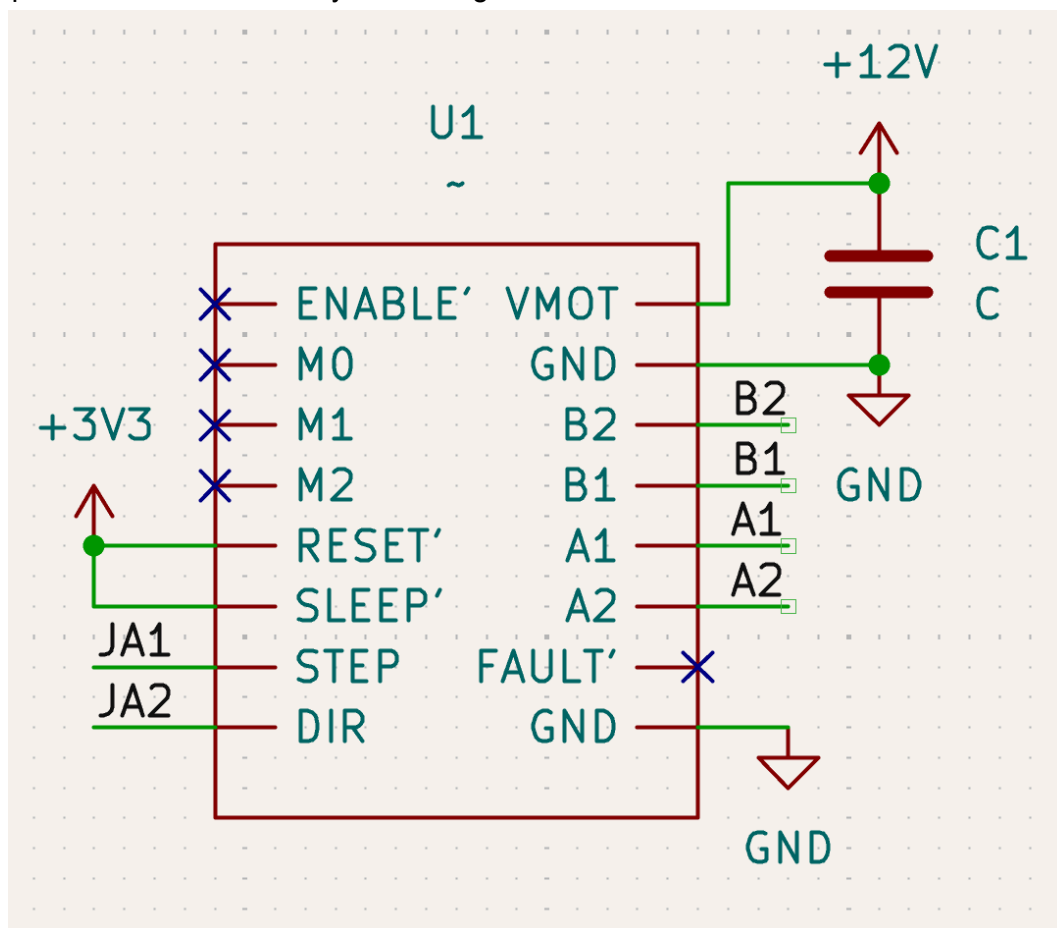


Figure 1: Stepper Motor Driver Schematic

Initially, one major struggle faced was the motor drivers not working. It appeared that all the pins were put in correctly, but one motor driver would work and heat up excessively while the other simply would not work. Upon discussion with another team and extensive research, it was discovered that the motor drivers had a built-in potentiometer to limit the current. The potentiometer's voltage was measured with a multimeter and adjusted to 0.8V, allowing both motor drivers to work effectively without overheating.

Another challenge faced was that, even though both drivers, both motors, and the code were verified to be working, only one would move at a time. It was discovered that the JA[3] pin was not working, and both motors were able to move once different JA pins were used. Chris Bingham was informed of this detail.

The servo has 3 pins: power, ground, and signal. The power and ground were connected via a breadboard to 3.3V and GND. The signal was connected to JB[1] on the FPGA. The schematic of the circuit used to control the servo is displayed in Figure 2. The servo takes in pulse width modulation (PWM) for precise motor control.

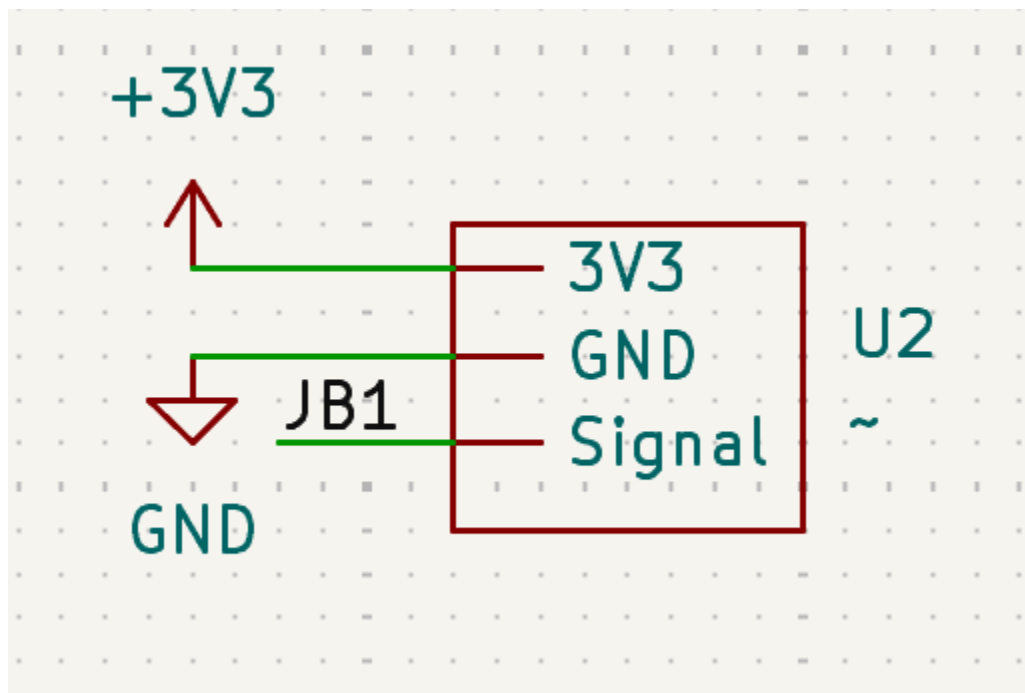


Figure 2: Servo Schematic

With more time, there were plans to make a screen so that the user could see what they were typing. This was planned to be implemented with a keyboard going into the lab PC with a Python app that sent data to the FPGA via Universal Asynchronous Receiver-Transmitter (UART) through the MicroUSB cable. Another idea was to make a phone app that would transmit data to the FPGA via Bluetooth.

Input Processing

When pressing a key, a scan code is sent from the keyboard. When the key is released, it sends the terminating byte 0xF0, followed by the scan code of the same released key. It also sends a read_data signal that is high when a key is being pressed.

In order to filter out these terminating bytes, the wrapper module of the Verilog checked every clock cycle for the read_data signal from the keyboard. If there was something to read, the wrapper checked whether or not it was a terminating byte. If it was not, it would take in the scan code of the key pressed. Otherwise, it would set an ignore wire high in order to ignore the scan code that is produced when the key is released so that the key is not duplicated.

One issue was that, with the original implementation of the input processing, the same character could not be pressed twice on the keyboard. To temporarily address this, the MIPS code blocked characters from being written twice. As the issue was being attempted to be fixed, there was a point when the same character would be written again and again when it was only pressed once. To discover where the issue was coming from, the LEDs were set to display the scan code data, other possible issues were ruled out, and iterative testing was done. Eventually, the issue was traced to the data being overwritten to 0 when the keyboard's read_data was low. Once this was fixed, the same key could be pressed multiple times to write it that number of times.

Motion Control

To control the step and direction pins with the FPGA, motion controller modules were developed in Verilog. When testing, it was found that the stepper motors work well when the step and direction inputs are pulsed at 500Hz, so a built in counter was developed to divide the clock to 500Hz. A register stores the current X and Y positions, and the module has an input of the target coordinates. Based on these values, the Verilog constantly checks for the target position relative to the current position. This would thus update the step, direction, and current position, until the target position is reached.

When the motors are moving, the module sets the busy wire high. Once this motion is completed, the module sets the busy wire low. The busy signal ensures that only one instruction is performed at a time. In order to begin a motion, the control signal must be set high from the MIPS code. The control signal acts as a start trigger to begin motor control.

To move the servo precisely with PWM, the PWMSerializer module given in Lab 9 was used to easily implement custom PWM signals. For this specific servo, a 20ms period was used with duty cycles of 1000μs and 2000μs to lift up and bring down the marker, respectively. The PWMSerializer module uses duty cycles in a range from 0 to 1023, so a simple ratio calculation was done to convert the duty cycles appropriately.

Character Generation

In order to simplify character generation, each character was given a predetermined square box of 200 by 200, where the unit is one step of the stepper motor. Each character was then written with lines parallel to the x-axis, y-axis, or $y=x$ line. This was because both stepper motors moved at the same speed, so lines of other slopes were not possible. However, if more time was given, the mode pins of the stepper motor would be used to draw lines of different slopes by controlling the step width of the motors, making the letters look more natural.

The entire alphabet was able to be written out by the machine. However, with more time, a simple addition would be to add numbers and punctuation marks as well.

MIPS

The MIPS integration of the project is based on a main loop checking for the desired location in memory, in which the scan code from the key pressed on the keyboard could be located. It loops around a large switch case for all 26 alphabet characters, where the cases are the scan codes. Once a valid scan code is seen, it jumps to a function coded to draw that character.

This function assigns target x and y coordinates to registers a0 and a1 respectively. After each coordinate is written, there is a jump to a draw line function. There, the values are saved in specific memory addresses, where the Verilog will adjust step and dir pulses until the current xy position equals the target xy position.

The z-axis motion is also triggered through MIPS; to move the pen up and down, the MIPS code saves a boolean value to memory, which the Verilog sees and sends the respective PWM to move the servo.

Initially, a challenge faced was that the servo took too long to move the pen up and down, so characters were not fully drawn out. To fix this, a large delay was implemented via addi, sll, bne instructions. This provided sufficient time for the servo to move the pen up or down before it moved in the xy plane again, neatening the characters.

After a character is written, it jumps to the next character function, which moves the marker slightly to the right for character margins. The origin is also shifted to the right. This shifting origin is what allows the character drawing functions to use generic coordinate mapping.

Other functionalities implemented using MIPS are the automatic newline, enter key, and reset key. Each line fits 6 characters, so one counter keeps track of the number of letters per line. If this limit of 6 characters is reached, it will auto-jump to the next line. When the user presses "Enter," this jump will be made manually. For the reset key, when the user presses the key "0" in the keyboard, the pen is moved back to its origin. These functions all move the origin accordingly, allowing characters to keep being written accurately.

System Integration

In order to integrate the assembly code, I/O, and Verilog code, specific addresses in memory were assigned to hold specific information. Table 1 shows what data is stored in which address.

Data Stored	Address
Scan Code	4098
Target X	4100
Target Y	4101
Motor Control	4102
Motor Status	4103
Pen Up	4104

Table 1: Data to Address Table

The scan code and motor status are inputs, so that by using the load word instruction, these values can be received from the wrapper. The other data are all outputs, so by using the store word instruction, the data goes from the MIPS code into the wrapper and motion controller modules. This is how target coordinates are set in MIPS and the wrapper uses them to move the stepper motors accordingly. It is also how a key is typed in the keyboard and the MIPS code can decide which character to draw based on it.

For the busy signal, or the motor status, the MIPS code checks for it every loop with a load word instruction. If the motors are moving, the MIPS code will not start writing a new character. The MIPS code also updates the control signal with a store word instruction so that the wrapper knows to begin moving the motors.

System Operation

In order to convert a keyboard press into organized motor and servo movements, a sequential data pipeline was implemented. When a key is pressed, the keyboard sends a scan code to the FPGA, which is stored at a specific memory-mapped address. The MIPS processor continuously polls this address to detect new key presses. Once a valid key is detected, the processor calls the corresponding character drawing function written in assembly, asserting the control signal to begin motion. The motion controller drives the motors while asserting the busy signal. The MIPS code polls this signal and waits until it is cleared before issuing the next command, ensuring sequential execution of movements. A diagram that displays how the whole process works is shown in Figure 3.

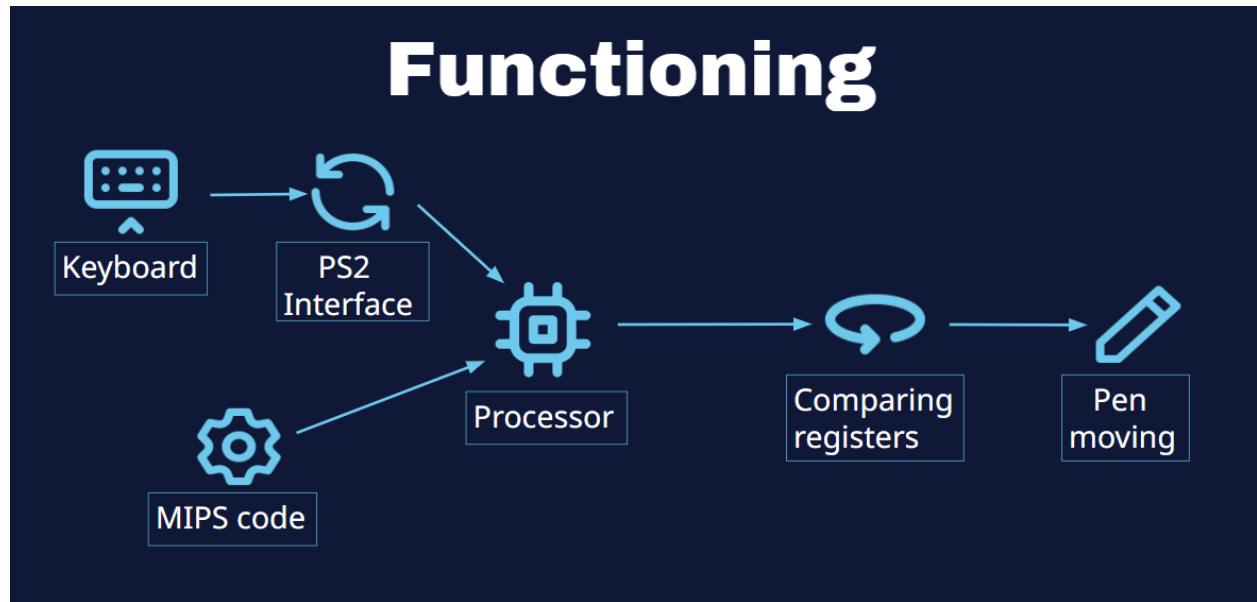


Figure 3: Diagram of System Overview

Mechanical Elements

The primary mechanical element was a gantry that allowed movement in the x and y directions. This was provided by Chris Bingham. It had a stepper motor for both axes, which used a system of pulleys to move the target in any direction. A wooden table was designed to hold the FPGA and wiring above the gantry. An image of the entire system can be seen in Figure 4.

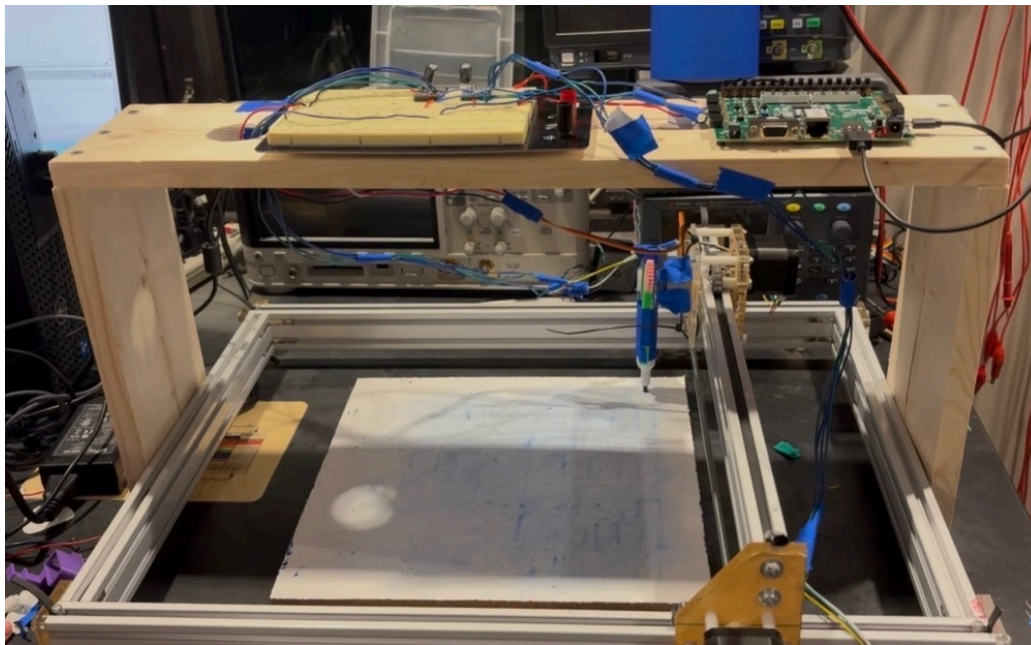


Figure 4: Machine Overview

In order to move the marker up and down, a separate system that controlled the z-axis needed to be designed. This was accomplished with a servo that moved a rack and pinion, turning angular movement into linear movement. A CAD of the design can be seen in Figure 5. The rack had a built-in marker holder to keep the rack steady in the movement. The servo mount had holes to zip tie it onto the stepper motor gantry and a holder to keep the rack steady during vertical movement.

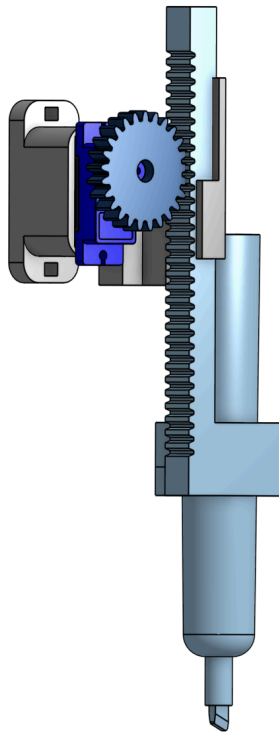


Figure 5: CAD of Servo and Rack and Pinion

The servo mount had various designs that had to be tested and iterated upon, as the rack would either be too close or too far from the gear. Some designs had the rack holder even break. An image of some of the implementations of the design is displayed in Figure 6. The arm was strengthened and the placement of the arm was more accurately defined.

Another addition was a marker holder in the rack. Initially, the marker was held onto the rack with some tape, resulting in uneven writing. By adding a tight holder, the marker was held in place effectively.

With more time, the servo mount would be improved to hold the servo in place better. This was because tape was needed to hold the servo in the mount as it jolted around and would pop out of the mount a bit, causing the rack and pinion system to be misaligned.

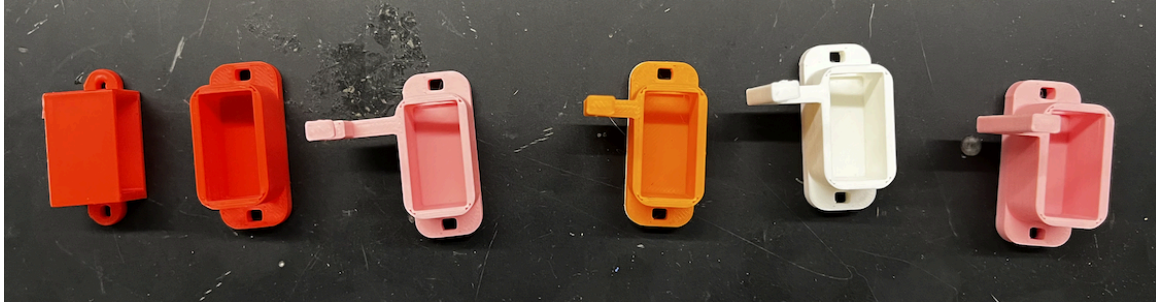


Figure 6: Various Designs of Servo Mount

Test Plan

All subsystems were validated individually and then as an integrated system. Details of the test plan are under each subsection, so a summary bulleted list is provided.

- Stepper motor movement
 - Used function generator with square waves
 - At 500Hz, motors rotated smoothly
- Stepper motor direction
 - Used power supply to set dir
 - At high dir, x moved right and y down. Thus, set origin as top left
- Scan code
 - Used FPGA LEDs to check correct scan codes were produced
 - All hex codes were accurate
- MIPS to move motors
 - Used sw instructions in MIPS to test simple strokes were writable
 - Simple lines in x and y could be written with the motor control modules
- Character writing
 - Checked that key press resulted in correct character being written
 - Every character was written accurately
- Rack and pinion
 - Checked that the servo turning the gear would produce vertical movement
 - Moving servo effectively moved pen up and down
- Vertical movement integration
 - Checked that vertical movement on each key was fluid and did not make the character write falsely.
 - Every character was still written accurately

Challenges

All of the challenges that were faced are written in detail in the subsections above. However, as a summary, a bulleted list of these challenges is provided:

- Current limit of motor drivers with built-in potentiometer
- JA[3] pin not working
- Unable to write same character twice in a row
- Servo took too long to move down and up
- Pulley system loose
- Rack and pinion too far apart
- Servo mount poor durability
- Marker taped on the rack

Future Improvements

Similar to the challenges section, ideas for future improvements are written in detail in the subsections above. However, as a summary, a bulleted list of them is provided:

- Adding move and beq instructions
- Seeing what you type
 - UART Python app through Desktop PC
 - Bluetooth phone app
 - Implementing a character buffer in MIPS
- Using mode pins for finer line control to have more natural characters
- Add numbers and punctuation
- Servo better fitted into the servo mount